

회전 / 직진 튜닝 가이드

기기마다 모터 편차·마찰·바퀴 그림이 달라서 같은 코드라도 회전각·직진성이 조금씩 다릅니다. 이 문서는 어떤 증상에 어떤 값을 만져야 하는지, **실제 어떻게 수정하면 되는지** 함수별 전체 코드와 함께 정리합니다.

각 코드블록 안의 `// [튜닝] ...` 주석이 붙은 줄의 숫자가 조정 대상입니다.

1. 회전 함수 3종 — 구조

PivotTurnLeft / PivotTurnRight ($\approx 90^\circ$)

각 회전은 2단계로 동작합니다:

1. **킥스타트** — 양쪽 동일 PWM 으로 정지마찰 극복 (50 ms)
2. **회전 본구간** — 한쪽 약(90), 한쪽 강(180) PWM 으로 시간만큼 회전. **시간이 회전각을 결정**합니다.

화물 적재 (`eWareHousePosition`) 시 더 무거우므로 같은 PWM 으로 더 오래 돌립니다 → **140 ms vs 비적재 110 ms**.

TurnHalf ($\approx 180^\circ$)

180° 한 번에 돌므로 본구간 한 단락. **시간(450 ms) 만** 조정하면 됩니다.

직진 / 라인트레이싱

`Power` (기본 110) 를 기준으로 P 제어로 좌/우 PWM 을 조정. `drive()` 안에서 EEPROM 모터 캘리브와 `SPEED_SCALE` 이 곱해집니다.

2. 핵심 튜닝 값 한눈에

값	위치	기본값	설명
회전 본구간 시간 (Pivot)	PivotTurnLeft / PivotTurnRight	140 / 110 ms	적재/비적재 본구간 delay. 회전각 조정의 1순위

값	위치	기본값	설명
회전 PWM 강쪽 (Pivot)	같은 함수 본구간	180	빠른 바퀴 PWM. 회전 속도 결정
회전 PWM 약쪽 (Pivot)	같은 함수 본구간	90	느린 바퀴 PWM. 값이 클수록 회전 반경 ↑
킵스타트 PWM (Pivot)	같은 함수 [1] 구간	170	정지마찰 극복용 초기 부스트
킵스타트 시간 (Pivot)	같은 함수 [1] 구간	50 ms	부스트 지속시간
180° 본구간 시간	TurnHalf	450 ms	TurnHalf 회전각
180° 본구간 PWM	같은 함수	170	TurnHalf 회전 속도
<code>SPEED_SCALE</code>	Controller.h:28	<code>0.6f</code>	전역 PWM/delay 스케일
<code>Power</code>	Controller.h:129	110	기본 라인트레이스 전진 PWM
<code>Kp</code>	Controller.h:133	0.05	라인 정렬 P 제어 비례 상수
<code>maxCorrection</code>	Controller.h:134	35.0	P 제어 1회 최대 보정값
<code>_motorCalibL/R</code>	EEPROM (+8/+12)	float	직진 보정 (별도 캘리브 스케치)
<code>LINEDETECT_THRESHOLD_MIN</code>	Controller.h:16	730	교차로 검출 임계값
<code>OBSTACLE_THRESHOLD</code>	Controller.h:23	500	전방중앙 IR 장애물 임계값

3. 증상 → 만질 값

증상	우선 조정	어디서
좌회전 부족 (90° 못 채움)	본구간 delay 110/140 ↑ (+10 ms)	4.1
좌회전 과회전	같은 값 ↓	4.1
우회전만 다름	우회전 본구간 delay 따로 조정	4.2
화물 들 때만 어긋남	<code>eWareHousePosition ? 140</code> 쪽만 조정	4.1 / 4.2
180° 어긋남	450 ms ↑/↓ (±30)	4.3
회전 시작 못 함	킵스타트 PWM 170 ↑ 또는 <code>SPEED_SCALE</code> ↑	4.1 / 4.2 / 4.5
회전 중 미끄러짐	킵스타트 시간 50 ms ↑ (+20)	4.1 / 4.2
직진 한쪽으로 휨	<code>_motorCalibL/R</code> (별도 캘리브 스케치)	EEPROM
라인 위 갈지자 흔들림	<code>Kp</code> ↓, <code>maxCorrection</code> ↓	4.5
P 제어 둔함	<code>Kp</code> ↑	4.5

증상	우선 조정	어디서
라인 인식 누락	<code>LINEDETECT_THRESHOLD_MIN</code> ↓	4.5
라인 외 잡음에 트립	<code>LINEDETECT_THRESHOLD_MIN</code> ↑	4.5
장애물 오감지	<code>OBSTACLE_THRESHOLD</code> ↓	4.5
장애물 미감지	<code>OBSTACLE_THRESHOLD</code> ↑	4.5
화물 적재 시 라인트레이스가 너무 빠름	<code>Power - 20</code> 의 20 을 ↑ (예: 30)	4.4

4. 조정 코드 예시 — 함수 전체

각 함수 안에서 `// [튜닝]` 주석이 붙은 줄의 숫자만 바꾸시면 됩니다. 다른 줄은 건드리지 마세요.

4.1 PivotTurnLeft — 좌회전 ($\approx 90^\circ$)

위치: [Controller.cpp:553](#)

```

void Controller::PivotTurnLeft()
{
    if (Serial) Serial.println("Enter Pivot turn Left");
    analogWrite(LeftWheelPWM, 0);
    analogWrite(RightWheelPWM, 0);
    delay(10);
    digitalWrite(LeftWheelDir, 0);
    digitalWrite(RightWheelDir, 0);

    Move();
    delay(10);
    // [1] 킥스타트: 정지 마찰 극복용 초기 부스트 (양쪽 동일 PWM, motorCalib 적용,
50ms)
    analogWrite(LeftWheelPWM, (int)(170 * _motorCalibL * SPEED_SCALE)); // [튜닝]
170: 회전 시작 못 함 → ↑ (예: 190)
    analogWrite(RightWheelPWM, (int)(170 * _motorCalibR * SPEED_SCALE)); // [튜닝]
170: 위 줄과 같은 값으로
    delay((unsigned long)(50 / SPEED_SCALE)); // [튜닝]
50ms: 부스트 시간. 미끄러지면 ↑ (예: 70)
    // [2] 회전 구간: 왼쪽 약, 오른쪽 강 → 좌회전 (시간으로 회전각 결정)
    analogWrite(LeftWheelPWM, (int)(90 * _motorCalibL * SPEED_SCALE)); // [튜닝]
90: 약쪽 PWM (회전 반경)
    analogWrite(RightWheelPWM, (int)(180 * _motorCalibR * SPEED_SCALE)); // [튜닝]
180: 강쪽 PWM (회전 속도). 만지면 아래 delay 재튜닝
    delay((unsigned long)((currentPosition == eWarehousePosition ? 140 : 110) /
SPEED_SCALE)); // [튜닝] 140/110: 적재/비적재 본구간 시간 - 좌회전 각도 1순위 (부족 ↑,
과회전 ↓)

    // [3] 정지
    analogWrite(LeftWheelPWM, 0);
    analogWrite(RightWheelPWM, 0);

    if (Serial) Serial.println("Leave Pivot turn Left");
}

```

4.2 PivotTurnRight — 우회전 (≈90°)

위치: [Controller.cpp:580](#)

```

void Controller::PivotTurnRight()
{
    if (Serial) Serial.println("Enter Pivot turn Right");
    analogWrite(LeftWheelPWM, 0);
    analogWrite(RightWheelPWM, 0);
    delay(10);
    digitalWrite(LeftWheelDir, 1);
    digitalWrite(RightWheelDir, 1);

    Move();
    delay(10);
    // [1] 킥스타트: 정지 마찰 극복용 초기 부스트 (양쪽 동일 PWM, 50ms)
    analogWrite(LeftWheelPWM, (int)(170 * _motorCalibL * SPEED_SCALE)); // [튜닝]
170: 회전 시작 못 함 → ↑ (예: 190)
    analogWrite(RightWheelPWM, (int)(170 * _motorCalibR * SPEED_SCALE)); // [튜닝]
170: 위 줄과 같은 값으로
    delay((unsigned long)(50 / SPEED_SCALE)); // [튜닝]
50ms: 부스트 시간
    // [2] 회전 구간: 왼쪽 강, 오른쪽 약 → 우회전 (시간으로 회전각 결정)
    analogWrite(LeftWheelPWM, (int)(170 * _motorCalibL * SPEED_SCALE)); // [튜닝]
170: 강쪽 PWM (회전 속도). 만지면 아래 delay 재튜닝
    analogWrite(RightWheelPWM, (int)(90 * _motorCalibR * SPEED_SCALE)); // [튜닝]
90: 약쪽 PWM (회전 반경)
    delay((unsigned long)((currentPosition == eWarehousePosition ? 140 : 110) /
SPEED_SCALE)); // [튜닝] 140/110: 적재/비적재 본구간 시간 - 우회전 각도 1순위 (부족 ↑,
과회전 ↓)

    // [3] 정지
    analogWrite(LeftWheelPWM, 0);
    analogWrite(RightWheelPWM, 0);
    if (Serial) Serial.println("Leave Pivot turn Right");
}

```

참고: 좌/우 회전 시간(140/110)을 각각 독립적으로 조정할 수 있습니다. 좌만 부족·우는 정상 이면 4.1 의 값만 바꾸세요.

4.3 TurnHalf — 180°

위치: [Controller.cpp:545](#)

C++

```

void Controller::TurnHalf() {
    drive(BACKWARD, 80, FORWARD, 80);           // [튜닝] 80: 부드러운 킥스타트
    PWM (보통 그대로)
    delay((unsigned long)(50 / SPEED_SCALE));    // [튜닝] 50ms: 킥스타트 시간
    drive(BACKWARD, 170, FORWARD, 170);         // [튜닝] 170: 본구간 PWM (회전
    속도). 만지면 아래 delay 재튜닝
    delay((unsigned long)(450 / SPEED_SCALE));   // [튜닝] 450ms: 180° 회전 시간
    - 부족 ↑ (±30), 과함 ↓
    Stop();
}

```

4.4 LineTrace — 직진 P 제어

위치: [Controller.cpp:428](#)

```

void Controller::LineTrace() {
    static uint8_t bSignalHigh = 0;
    static unsigned long lastSensorLog = 0;

    int leftRaw = GetLeft();
    int rightRaw = GetRight();

    // 디버그: 200ms마다 좌/우 raw 값과 정규화 값 출력
    if (millis() - lastSensorLog > 200) {
        Serial.print("L_raw="); Serial.print(leftRaw);
        Serial.print(" R_raw="); Serial.print(rightRaw);
        Serial.print(" | L_n="); Serial.print(normalizeLeft(leftRaw));
        Serial.print(" R_n="); Serial.println(normalizeRight(rightRaw));
        lastSensorLog = millis();
    }

    // 교차로(검은선 2개 동시) 판단
    if (rightRaw > LINEDETECT_THRESHOLD_MIN && leftRaw > LINEDETECT_THRESHOLD_MIN) {
        if (bSignalHigh == 0) {
            nLineCounter++;
            if (Serial) {
                Serial.println(String("LINE!!! :") + String(nLineCounter));
            }
            bSignalHigh = 1;
        }

        Forward(Power);
        delay((unsigned long)(50 / SPEED_SCALE)); // [튜닝] 50ms: 교차로 통과 후 살짝
        더 전진하는 시간 (정밀 정렬 시 후진으로 라인 다시 찾기 위함)
    }
    else {
        if (bSignalHigh) {
            bSignalHigh = 0;
        }

        // 부드러운 P-제어
        int leftNorm = normalizeLeft(leftRaw);
        int rightNorm = normalizeRight(rightRaw);

        int error = rightNorm - leftNorm;
        float correction = Kp * error; // Kp 는
        Controller.h 에서 조정 (4.5 참조)

        if (correction > maxCorrection) correction = maxCorrection; // maxCorrection
        도 Controller.h
        if (correction < -maxCorrection) correction = -maxCorrection;

        int basePower = (currentPosition == eWarehousePosition) ? Power - 20 : Power;
    }
}

```

```
// [튜닝] 20: 적재 시 감속폭. 더 천천히 → ↑ (예: 30~40)
```

```
float leftPower = basePower + correction;
float rightPower = basePower - correction;

drive(FORWARD, (int)leftPower, FORWARD, (int)rightPower);
}
}
```

4.5 Controller.h — 전역 상수 & 멤버 변수

위치: [Controller.h](#)

C++

```
// === 임계값 & 전역 스케일 (Controller.h 상단) ===
```

```
#define LINEDETECT_THRESHOLD_MIN 730 // [튜닝] 교차로 검출 임계값 (정규화 raw 기준)
// 라인 인식 누락 → ↓ / 잡음 트립 → ↑
// 흰 raw 와 검은 raw 의 중간 위쪽으로
```

잡기

```
#define OBSTACLE_THRESHOLD 500 // [튜닝] IR 장애물 임계값 (A0 raw < 이 값일 때
트립)
// 오감지(없는데 트립) → ↓ / 미감지 → ↑
```

```
#define SPEED_SCALE 0.6f // [튜닝] 전역 PWM/delay 스케일 (0.5 ~ 1.0)
// 낮춤 → 천천히, 단 0.5 미만은 정지마찰
```

못이김

```
// 바꾸면 4.1~4.3 의 delay 들 재튜닝 필요
```

```
// === Controller 클래스 멤버 변수 (Controller.h 클래스 안) ===
```

```
int Power = 110; // [튜닝] 기본 라인트레이스 전진 PWM
// 빠르게 → ↑ (Kp 도 같이 재튜닝 권장)
```

```
// 💡 첫 번째 코드에서 가져온 P제어용 변수
```

```
float Kp = 0.05; // [튜닝] P 제어 비례 상수
// 갈지자 ↓ (예: 0.03) / 둔감 ↑ (예:
0.07)
```

```
float maxCorrection = 35.0; // [튜닝] 1회 보정 한도
// 흔들림 ↓ (예: 25) / 급커브 적응 ↑
```

(예: 50)

5. 권장 튜닝 순서

1. EEPROM 모터·센서 캘리브 먼저

- 워크스페이스의 별도 스케치 ([Calibration.ino](#), [MotorCalibration.ino](#)) 로 흑/백 raw 값과 모터 calib 을 EEPROM 240번지 이후에 저장.
- 본 스케치는 시작 시 [readData\(\)](#) 로 이걸 읽어 사용 — 0 으로 비어 있으면 직진/정규화가 망가짐.

2. [SPEED_SCALE](#) 결정 (§4.5)

- 모터가 안정적으로 돌면서 너무 빠르지 않은 최저값 (보통 0.5 ~ 0.7).
- 한 번 정하면 고정. 바꾸면 회전 delay 전부 재튜닝.

3. 직진성 점검 (§4.4, §4.5)

- 라인 위 갈지자 없이 진행되는지. 필요 시 [Kp](#), [maxCorrection](#), [Power](#) 조정.

4. PivotTurn 좌·우 각각 (§4.1, §4.2)

- 직진 → 정지 → PivotTurn 한 번 → 직진 시퀀스. 90° 되는지 바닥에 직각 테이프로 확인.
- 적재(140 ms)와 비적재(110 ms) 각각 별도 조정.

5. TurnHalf 180° (§4.3)

- 마지막. ±30 ms 단위로 잡으면 충분.

6. EEPROM 캘리브레이션 상세

[readData\(\)](#) 가 EEPROM 240번지부터 6개 값을 읽어옵니다:

Offset	Type	필드	의미
+0	int16	_rightWhite	오른쪽 바닥센서 흰색 raw
+2	int16	_leftWhite	왼쪽 바닥센서 흰색 raw
+4	int16	_rightBlack	오른쪽 바닥센서 흑색(라인) raw
+6	int16	_leftBlack	왼쪽 바닥센서 흑색 raw
+8	float	_motorCalibR	오른쪽 PWM 배수
+12	float	_motorCalibL	왼쪽 PWM 배수

- **모터 calib 값:** 보통 **0.95 ~ 1.05** 범위. 1.0 = 그대로. 한쪽이 빠르면 그쪽 값을 낮춤.
- **센서 흑/백 값:** [normalizeLeft/Right\(\)](#) 가 0~1000 정규화에 사용. **흰 raw < 검은 raw** 여야 정상. 뒤집혀 있으면 정규화가 한쪽으로 항상 클램프되어 P 제어가 한쪽으로만 꺾이는 증상이 납니다 ([CLAUDE.md](#) 의 캘리브 이력 참조).
- 본 스케치 플래시는 EEPROM 을 지우지 않지만, **스톡 펌웨어 플래시는 지울 수 있으니** 캘리브 후엔 이 스케치만 올리도록 주의.

7. SPEED_SCALE 주의사항

[Controller.h:28](#) `SPEED_SCALE = 0.6f`. 거의 모든 곳에 적용됩니다:

- 모든 PWM 출력 $\times$ `SPEED_SCALE` (느려짐)
- 거의 모든 delay $\times$ $1/\text{SPEED_SCALE}$ (시간 늘림, 같은 각도/거리 유지)
- 예외: 후진 정렬 dance ([LineTracer](#) 의 400 ms 후진) 는 의도적으로 $\times$ `SPEED_SCALE` (시간 안 늘리고 줄임). 관성 거리 $\propto v^2 \propto \text{PWM}^2$ 보정 때문.

`SPEED_SCALE` 을 바꾸면 회전 각도가 미세하게 변합니다 — 정지마찰·관성이 PWM 에 비선형이라 시간 보정만으로 정확히 같은 각도가 안 나옵니다. 가급적 한 번 정하고 그 위에서 다른 값들을 잡으세요.

8. 디버그 출력 활용

본 스케치는 9600 baud Serial 로 다음을 출력합니다 — Arduino IDE Serial Monitor (또는 `arduino-cli monitor -c baudrate=9600`) 로 보세요.

- [LineTrace\(\)](#) 가 200 ms 마다 좌/우 raw + 정규화 값:

```
L_raw=320 R_raw=423 | L_n=0 R_n=12
```

C++

raw 값으로 EEPROM 캘리브 일치 여부 확인. 정규화 값이 항상 0 또는 1000 으로 클램프되면 캘리브 뒤집힘 의심.

- 각 회전 진입/이탈: `Enter Pivot turn Left` / `Leave Pivot turn Right` 등.
 - 장애물 감지: `Obstacle! Backing to prev node.`
 - 동적 차단 등록: `Dyn-blocked cell (x,y) count=N`
 - Dead-end 백트래킹: `Dead-end at (x,y) - backtracking`
 - 네비게이션 진행: `Nav: (x1,y1) -> (x2,y2)`
-

9. 마지막 체크리스트 — 회전이 자꾸 어긋날 때

- 배터리 전압 충분한가? 낮으면 모터 토크 떨어져 회전 부족.
- 바퀴·모터 마운트가 흔들리지 않는가? 슬립 발생 시 회전각 들쭉날쭉.
- 바닥 마찰이 일정한가? 매끈한 바닥과 카펫에서 회전각 다름. 캘리브 환경과 운영 환경 일치시키기.
- 정밀 정렬은 $y=0/7$ 에서만 작동합니다 ([LineTracer](#) `_preciseRealign`). 중간 행에서는 미세 어긋남 누적 가능 — 누적이 크면 더 잦은 정밀 정렬을 고려.

5. `_motorCalibL/R` 이 **NaN / 0** 이 아닌가? EEPROM 미초기화 보드면 그럴 수 있음. 시작 시 Serial로 찍히는 `rW lW rB lB` 값이 정상 범위인지 첫 줄에서 확인.